



CIL-- 합성을 통한 Sparrow 분석기 공격

정원준

2026년 7월 16일

ROPAS@SNU

목차

정적 분석기 공격

Sparrow 분석기 공격 목표

CIL-- 언어

Sparrow의 실행의미와 대응시키기

메모리를 포함하여 합성하는 방법

앞으로 할 일

정적 분석기 공격

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

■ x가 어떤 값을 가질 수 있을까?

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?
- 프로그램을 돌려보지 않고 미리 알 수는 없을까?

정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?
- 프로그램을 돌려보지 않고 미리 알 수는 없을까?
- 완벽하게 알 수는 없음!

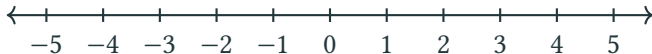
정적 분석이란?

```
x = 1
while x < 10000:
    x = x + 1
print(x)
```

- x가 어떤 값을 가질 수 있을까?
- x가 특정 위치에서 어떤 값을 가질 수 있을까?
- 프로그램을 돌려보지 않고 미리 알 수는 없을까?
- 완벽하게 알 수는 없음!
- “Rice’s Theorem”

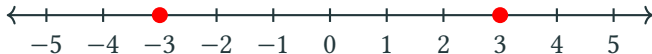
넓게 어림잡기

```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



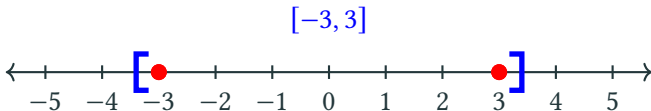
넓게 어림잡기

```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



넓게 어림잡기

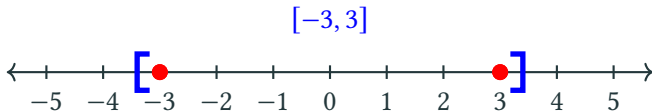
```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



공격 프로그램의 예시

공격 프로그램의 예시

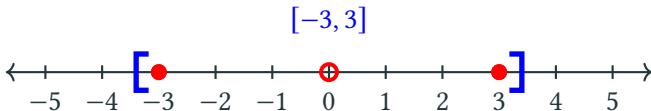
```
if x > 0:
    x = 3
else:
    x = -3
print(x)
```



공격 프로그램의 예시

```
if x > 0:  
    x = 3  
else:  
    x = -3  
y = 10 / x
```

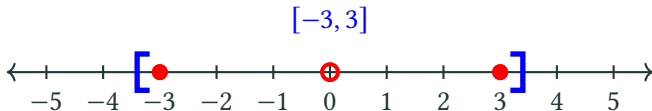
false alarm 발생



공격 프로그램의 예시

```
if x > 0:
    x = 3
else:
    x = -3
y = 10 / x
```

false alarm 발생



이런 공격을 일반적인 분석기에 대해서 하는 것이 목표

공격의 기대효과

공격의 기대효과

- 분석기의 약점을 파악하고 강화하는 가이드라인으로 사용

공격의 기대효과

- 분석기의 약점을 파악하고 강화하는 가이드라인으로 사용
- 분석이 잘 안 되는 코드 조각을 삽입해서 난독화로 사용

Sparrow 분석기 공격 목표

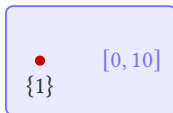
Sparrow 공격 목표

■ 정밀도 공격:

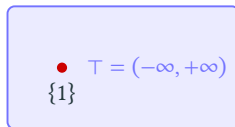
abstract semantics가 concrete semantics를 너무 크게 포함함

$$\{1\} = [1, 1]$$

concrete = abstract



concrete $\subsetneq$ abstract



abstract = $\top$

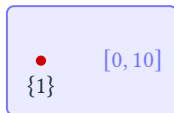
Sparrow 공격 목표

■ 정밀도 공격:

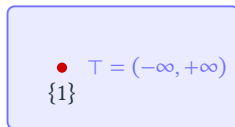
abstract semantics가 concrete semantics를 너무 크게 포함함



concrete = abstract



concrete $\subsetneq$ abstract



abstract = $\top$

■ soundness 공격:

concrete semantics가 abstract semantics에 포함되지 않음(버그)



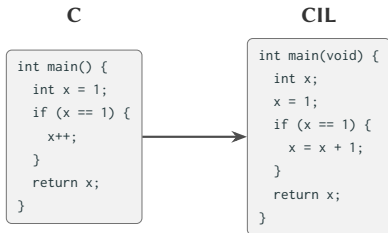
concrete $\not\subseteq$ abstract

Sparrow의 기본 구조

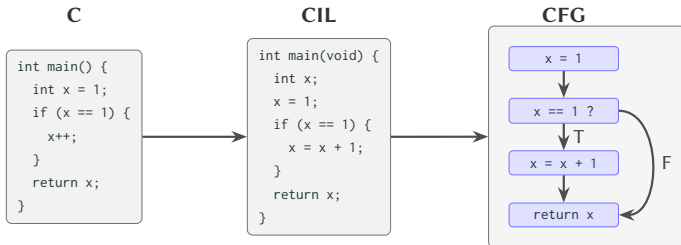
C

```
int main() {  
    int x = 1;  
    if (x == 1) {  
        x++;  
    }  
    return x;  
}
```

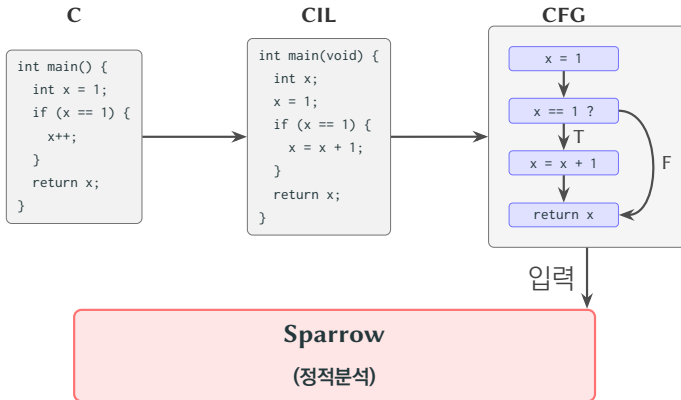
Sparrow의 기본 구조



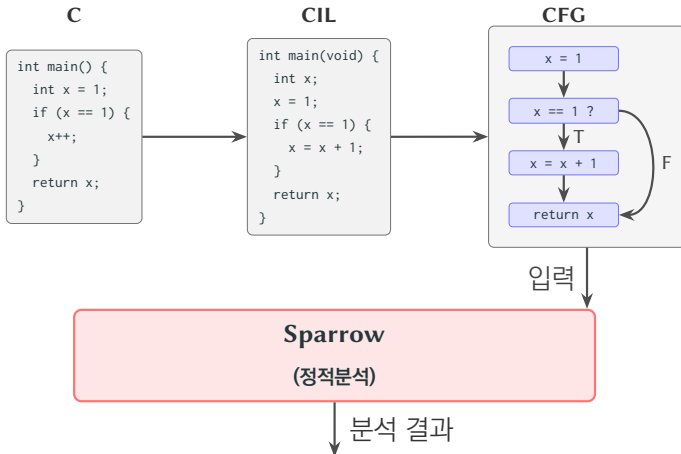
Sparrow의 기본 구조



Sparrow의 기본 구조



Sparrow의 기본 구조



CIL-- 언어

CIL 언어

C

```
int main() {  
    int sum = 0;  
    for (int i = 1; i <= n; i++)  
        sum += i;  
    return sum;  
}
```



CIL

```
int main(void) {  
    int sum;  
    int i;  
    sum = 0;  
    i = 1;  
    loop {  
        if (i > n) break;  
        sum = sum + i;  
        i = i + 1;  
    }  
    return sum;  
}
```

- C의 부분집합 — 분석을 쉽게 할 수 있는 언어

CIL 언어

C

```
int main() {  
    int sum = 0;  
    for (int i = 1; i <= n; i++)  
        sum += i;  
    return sum;  
}
```



CIL

```
int main(void) {  
    int sum;  
    int i;  
    sum = 0;  
    i = 1;  
    loop {  
        if (i > n) break;  
        sum = sum + i;  
        i = i + 1;  
    }  
    return sum;  
}
```

- C의 부분집합 — 분석을 쉽게 할 수 있는 언어
- side effect 제거

CIL 언어

C

```
int main() {  
    int sum = 0;  
    for (int i = 1; i <= n; i++)  
        sum += i;  
    return sum;  
}
```



CIL

```
int main(void) {  
    int sum;  
    int i;  
    sum = 0;  
    i = 1;  
    loop {  
        if (i > n) break;  
        sum = sum + i;  
        i = i + 1;  
    }  
    return sum;  
}
```

- C의 부분집합 – 분석을 쉽게 할 수 있는 언어
- side effect 제거
- 함수 호출은 그 자체로 한 statement

CIL 언어

C

```
int main() {  
    int sum = 0;  
    for (int i = 1; i <= n; i++)  
        sum += i;  
    return sum;  
}
```



CIL

```
int main(void) {  
    int sum;  
    int i;  
    sum = 0;  
    i = 1;  
    loop {  
        if (i > n) break;  
        sum = sum + i;  
        i = i + 1;  
    }  
    return sum;  
}
```

- C의 부분집합 – 분석을 쉽게 할 수 있는 언어
- side effect 제거
- 함수 호출은 그 자체로 한 statement
- ...

CIL-- 언어

- completeness 공격: 튜링 완전한 subset이기만 하면 충분 (Rice Theorem 덕분)

CIL-- 언어

- completeness 공격: 튜링 완전한 subset이지만 하면 충분 (Rice Theorem 덕분)
- soundness 공격: feature가 많을수록 좋음 (버그를 찾는 것이므로)

CIL-- 언어

- completeness 공격: 튜링 완전한 subset이지만 하면 충분 (Rice Theorem 덕분)
- soundness 공격: feature가 많을수록 좋음 (버그를 찾는 것이므로)
- CIL- : 의미적으로 관심 없는 것 제거 — 확장 계획도 없음 (typedef, struct 등)

CIL-- 언어

- completeness 공격: 튜링 완전한 subset이지만 하면 충분 (Rice Theorem 덕분)
- soundness 공격: feature가 많을수록 좋음 (버그를 찾는 것이므로)
- CIL- : 의미적으로 관심 없는 것 제거 — 확장 계획도 없음 (typedef, struct 등)
- CIL-- : 일단 미니멀하게 만들기 위해 이것저것 제거 — 나중에 추가 희망 (포인터 · malloc/free, 배열, int 외 타입)

Sparrow의 실행의미와 대응시키기

실행의미가 잘 정의되는가?

- 가장 큰 문제: Sparrow는 concrete semantics를 정확히 정의하지
않음

실행의미가 잘 정의되는가?

- 가장 큰 문제: Sparrow는 concrete semantics를 정확히 정의하지
않음
 - ▶ C: ISO C 표준이 있지만 자연어로 되어 있어서 엄밀하진 않음

실행의미가 잘 정의되는가?

- 가장 큰 문제: Sparrow는 concrete semantics를 정확히 정의하지 않음
 - ▶ C: ISO C 표준이 있지만 자연어로 되어 있어서 엄밀하진 않음
 - ▶ CIL: 마찬가지로 엄밀한 실행의미가 없음

실행의미가 잘 정의되는가?

- 가장 큰 문제: Sparrow는 concrete semantics를 정확히 정의하지
않음
 - ▶ C: ISO C 표준이 있지만 자연어로 되어 있어서 엄밀하진 않음
 - ▶ CIL: 마찬가지로 엄밀한 실행의미가 없음
 - ▶ Sparrow: 아예 없음

실행의미가 잘 정의되는가?

- 가장 큰 문제: Sparrow는 concrete semantics를 정확히 정의하지 않음
 - ▶ C: ISO C 표준이 있지만 자연어로 되어 있어서 엄밀하진 않음
 - ▶ CIL: 마찬가지로 엄밀한 실행의미가 없음
 - ▶ Sparrow: 아예 없음
 - ▶ 공격기: CIL--에 대한 big-step 나무구조와 증명 생성기는 있음

실행의미가 잘 정의되는가?

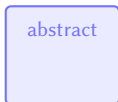
- 가장 큰 문제: Sparrow는 concrete semantics를 정확히 정의하지 않음
 - ▶ C: ISO C 표준이 있지만 자연어로 되어 있어서 엄밀하진 않음
 - ▶ CIL: 마찬가지로 엄밀한 실행의미가 없음
 - ▶ Sparrow: 아예 없음
 - ▶ 공격기: CIL--에 대한 big-step 나무구조와 증명 생성기는 있음
- 그렇다면, 공격기의 concrete 실행의미가 Sparrow의 abstract 실행의미에 포함되지 않을 때 “soundness가 깨졌다”고 말할 수 있는가?

실행의미가 잘 정의되는가?

- 가장 큰 문제: Sparrow는 concrete semantics를 정확히 정의하지 않음
 - ▶ C: ISO C 표준이 있지만 자연어로 되어 있어서 엄밀하진 않음
 - ▶ CIL: 마찬가지로 엄밀한 실행의미가 없음
 - ▶ Sparrow: 아예 없음
 - ▶ 공격기: CIL--에 대한 big-step 나무구조와 증명 생성기는 있음
- 그렇다면, 공격기의 concrete 실행의미가 Sparrow의 abstract 실행의미에 포함되지 않을 때 “soundness가 깨졌다”고 말할 수 있는가?
- 각종 비결정성을 모두 제거해야 제대로 공격 가능

비결정성을 제거하지 않으면...

- 공격을 성공했다고 생각했는데...

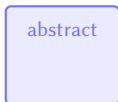


⇒ soundness 깨짐 (공격 성공!)

concrete

비결정성을 제거하지 않으면...

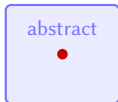
- 공격을 성공했다고 생각했는데...



● ⇒ soundness 깨짐 (공격 성공!)

concrete

- 사실 그건 비결정성 때문이었다고?!



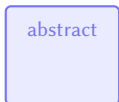
● ⇒ soundness 깨지지 않음.

Sparrow-concrete

Attack-concrete

비결정성을 제거하지 않으면...

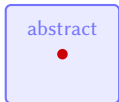
- 공격을 성공했다고 생각했는데...



● ⇒ soundness 깨짐 (공격 성공!)

concrete

- 사실 그건 비결정성 때문이었다고?!



● ⇒ soundness 깨지지 않음.

Sparrow-concrete Attack-concrete

- 따라서 모든 비결정성을 제거해야 올바른 공격을 할 수 있다.

C 언어의 비결정성

예	C	CIL	CIL--
h(f(),g()) f()+g() {f(),g()}	계산 순서 비결정	함수 호출은 expression으 로 들어가지 않음	OK
sizeof(int (*)(n++))	n++을 실행하거나/하지 않거나	n++ 문법이 없음	OK
i = i++ + 1;	Undefined Behavior	i++ 문법이 없음	OK
int x; use(x);	indeterminate value	indeterminate value	합성 시 생성 x
"a" == "a"	true/false 모두 가능	여전히 비결정적	string 없음
char c = -1;	-1/255	여전히 비결정적	char 없음
x >> 1	x < 0 일 때 UB	여전히 비결정적	비트연산 없음
malloc(n)	메모리 주소가 비결정적	fresh object	포인터 없음

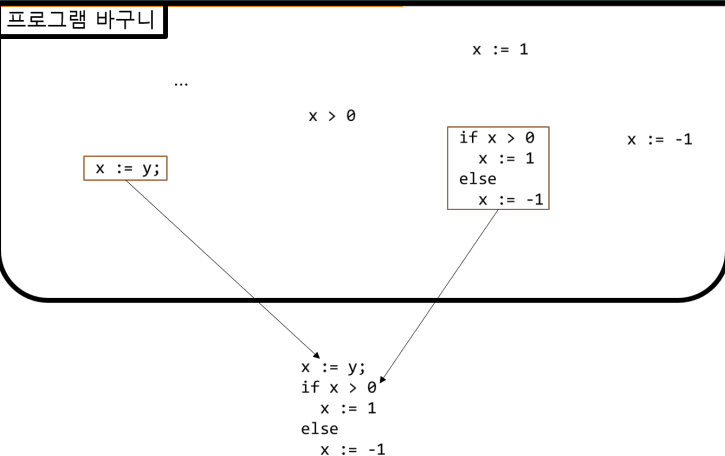
C 언어의 비결정성

예	C	CIL	CIL--
h(f(),g()) f()+g() {f(),g()}	계산 순서 비결정	함수 호출은 expression으 로 들어가지 않음	OK
sizeof(int (*)(n++))	n++을 실행하거나/하지 않거나	n++ 문법이 없음	OK
i = i++ + 1;	Undefined Behavior	i++ 문법이 없음	OK
int x; use(x);	indeterminate value	indeterminate value	합성 시 생성 x
"a" == "a"	true/false 모두 가능	여전히 비결정적	string 없음
char c = -1;	-1/255	여전히 비결정적	char 없음
x >> 1	x < 0 일 때 UB	여전히 비결정적	비트연산 없음
malloc(n)	메모리 주소가 비결정적	fresh object	포인터 없음

이외에도 malloc 실패 여부, struct의 byte 패딩, signed 정수 overflow 등 여러 가지 고려해야 할 비결정성이 많음

메모리를 포함하여 합성하는 방법

증명나무 합성과 메모리



- 프로그램 합성 대신 증명나무 합성 방식을 사용

증명나무 합성과 메모리

증명나무 바꾸니

...

```
-----
{x: 1} |- x => 1   {x: 1} |- 1 => 1
-----
{x: 1} |- (x < 1) => 0
```

```
-----
{} |- 1 => 1
-----
{} |- x := 1 => {x: 1}
```

```
-----
{x: 1} |- x => 1   {x: 1} |- 1 => 1
-----
{x: 1} |- (x < 1) => 0
-----
{x: 1} |- while (x < 1) => {x: 1}
      x := (x + 1)
```

```
-----
{} |- 1 => 1
```

```
-----
{x: 1} |- x => 1   {x: 1} |- 1 => 1
-----
{x: 1} |- (x < 1) => 0
```

- 프로그램 합성 대신 증명나무 합성 방식을 사용
- 실행 의미가 존재하는 프로그램만을 뽑아내기 위함!

```
while (x < 1)
  x := (x + 1)
```

증명나무 합성과 메모리

- 프로그램 합성 대신 증명나무 합성 방식을 사용
- 실행 의미가 존재하는 프로그램만을 뽑아내기 위함!
- 증명나무에는 항상 메모리가 따라 붙음

메모리는 코드를 따라 자연스럽게 나오는 것이 아닌가?

- 완전한 프로그램은 그 자체로 메모리를 결정할 수 있음

```
int main() {  
    int x; int y;  
    x = 1;  
    y = x + 1;  
    return x;  
}
```

메모리는 코드를 따라 자연스럽게 나오는 것이 아닌가?

- 완전한 프로그램은 그 자체로 메모리를 결정할 수 있음

```
int main() {  
    int x; int y;  
    x = 1;  
    y = x + 1;  
    return x;  
}
```

- 반면 부분 프로그램은 앞뒤 메모리의 관계만 알 수 있음

(전 메모리) → y = x + 1; → (후 메모리)

메모리를 최대한 나중에 결정하려고 하면...

$x = 1$; 증명나무

 $\{\} \mid - x = 1; \Rightarrow \{x \mid -> 1\}$

메모리를 최대한 나중에 결정하려고 하면...

$x = 1$; 증명나무

{ } |- $x = 1$; $\Rightarrow$ { x |-> 1 }

$y = x + 1$; 증명나무

{ x |-> ? } |- $x \Rightarrow ?$? + 1 $\Rightarrow ? + 1$

{ x |-> ? } |- $y = x + 1$; $\Rightarrow$ { x |-> ?, y |-> ? + 1 }

메모리를 최대한 나중에 결정하려고 하면...

$x = 1$; 증명나무

 $\{\} \vdash x = 1; \Rightarrow \{x \mapsto 1\}$

$y = x + 1$; 증명나무

 $\{x \mapsto ?\} \vdash x \Rightarrow ? \quad ? + 1 \Rightarrow ? + 1$

 $\{x \mapsto ?\} \vdash y = x + 1; \Rightarrow \{x \mapsto ?, y \mapsto ? + 1\}$

 $\{x \mapsto 1\} \vdash x \Rightarrow 1 \quad 1 + 1 \Rightarrow 2$

 $\{\} \vdash x = 1; \Rightarrow \{x \mapsto 1\} \quad \{x \mapsto 1\} \vdash y = x + 1; \Rightarrow \{x \mapsto 1, y \mapsto 2\}$

 $\{\} \vdash x = 1; y = x + 1; \Rightarrow \{x \mapsto 1, y \mapsto 2\}$

합친 증명나무

메모리를 최대한 나중에 결정하려고 하면...

$x = 1$; 증명나무

 $\{\} \vdash x = 1; \Rightarrow \{x \vdash 1\}$

$y = x + 1$; 증명나무

 $\{x \vdash ?\} \vdash x \Rightarrow ? \quad ? + 1 \Rightarrow ? + 1$

 $\{x \vdash ?\} \vdash y = x + 1; \Rightarrow \{x \vdash ?, y \vdash ? + 1\}$

 $\{x \vdash 1\} \vdash x \Rightarrow 1 \quad 1 + 1 \Rightarrow 2$

 $\{\} \vdash x = 1; \Rightarrow \{x \vdash 1\} \quad \{x \vdash 1\} \vdash y = x + 1; \Rightarrow \{x \vdash 1, y \vdash 2\}$

 $\{\} \vdash x = 1; y = x + 1; \Rightarrow \{x \vdash 1, y \vdash 2\}$

합친 증명나무

어 되나?

하지만 조건문, 반복문을 만나면...

```
loop{if(x == 0) break; x = x - 1;}
```

메모리를 모른 채로, 이 부분 프로그램의 증명나무를 그릴 수 있을까?

하지만 조건문, 반복문을 만나면...

```
loop{if(x == 0) break; x = x - 1;}
```

메모리를 모른 채로, 이 부분 프로그램의 증명나무를 그릴 수 있을까?

$x = 1$ 일 때

```
.....  
{x |> 0} |- if (x == 0) break; x = x - 1; => {x |> 0}, break  
.....  
{x |> 1} |- if (x == 0) break; x = x - 1; => {x |> 0} {x |> 0} |- loop { if (x == 0) break; x = x - 1; }; => {x |> 0}  
.....  
{ } |- x = 1; => {x |> 1} {x |> 1} |- loop { if (x == 0) break; x = x - 1; }; => {x |> 0}  
.....  
{ } |- x = 1; loop { if (x == 0) break; x = x - 1; }; => {x |> 0}
```

하지만 조건문, 반복문을 만나면...

```
loop{if(x == 0) break; x = x - 1;}
```

메모리를 모른 채로, 이 부분 프로그램의 증명나무를 그릴 수 있을까?

$x = 1$ 일 때

```

(x |> 0) |- if (x == 0) break; x = x - 1; => (x |> 0), break
-----
(x |> 1) |- if (x == 0) break; x = x - 1; => (x |> 0) (x |> 0) |- loop { if (x == 0) break; x = x - 1; }; => (x |> 0)
-----
{} |- x = 1; => (x |> 1) (x |> 1) |- loop { if (x == 0) break; x = x - 1; }; => (x |> 0)
-----
{} |- x = 1; loop { if (x == 0) break; x = x - 1; }; => (x |> 0)

```

$x = 2$ 일 때

```

(x |> 0) |- if (x == 0) break; x = x - 1; => (x |> 0), break
-----
(x |> 1) |- if (x == 0) break; x = x - 1; => (x |> 0) (x |> 0) |- loop { if (x == 0) break; x = x - 1; }; => (x |> 0)
-----
(x |> 2) |- if (x == 0) break; x = x - 1; => (x |> 1) (x |> 1) |- loop { if (x == 0) break; x = x - 1; }; => (x |> 0)
-----
{} |- x = 2; => (x |> 2) (x |> 2) |- loop { if (x == 0) break; x = x - 1; }; => (x |> 0)
-----
{} |- x = 2; loop { if (x == 0) break; x = x - 1; }; => (x |> 0)

```

하지만 조건문, 반복문을 만나면...

```
loop{if(x == 0) break; x = x - 1;}
```

메모리를 모른 채로, 이 부분 프로그램의 증명나무를 그릴 수 있을까?

$x = 1$ 일 때

```

{ } |- x = 1; loop { if (x == 0) break; x = x - 1; } => { x |-> 0 }, break
{ x |-> 1 } |- if (x == 0) break; x = x - 1; => { x |-> 0 }
{ x |-> 1 } |- loop { if (x == 0) break; x = x - 1; }; => { x |-> 0 }
{ } |- x = 1; loop { if (x == 0) break; x = x - 1; }; => { x |-> 0 }

```

$x = 2$ 일 때

```

{ } |- x = 2; loop { if (x == 0) break; x = x - 1; } => { x |-> 0 }, break
{ x |-> 2 } |- if (x == 0) break; x = x - 1; => { x |-> 1 }
{ x |-> 2 } |- loop { if (x == 0) break; x = x - 1; }; => { x |-> 0 }
{ } |- x = 2; loop { if (x == 0) break; x = x - 1; }; => { x |-> 0 }

```

조립이 잘 안 됨!

하지만 조건문, 반복문을 만나면...

- 메모리를 후처리를 한다고 해도 무한히 돌거나 정지 문제를 해결해야 하는 딜레마에 빠짐.

하지만 조건문, 반복문을 만나면...

- 메모리를 후처리를 한다고 해도 무한히 돌거나 정지 문제를 해결해야 하는 딜레마에 빠짐.
- 따라서 메모리는 나중에 계산하면 안 되고, 작은 증명나무를 만들 때 전후 메모리가 확실하게 정해져야 함.

합성 시 메모리 관리

- 실제 C 언어에서는 주소 연산을 통해 인접한 메모리를 마구 참조 가능하다.

합성 시 메모리 관리

- 실제 C 언어에서는 주소 연산을 통해 인접한 메모리를 마구 참조 가능하다.
- 특히 struct를 보면 적당히 포인터 + 1 연산을 이용해 인접 변수에 접근 가능.

```
1  #include <stdio.h>
2
3  struct S { int i; char c; };
4
5  int main(void) {
6      struct S s = {65, 'A'};
7      printf("before: s.c = %c\n", s.c);
8
9      int *p = &s.i;    // i를 가리키는 포인터
10     p = p + 1;        // 포인터 + 1: 다음 "슬롯"으로 이동 (실제로는 c의 위치)
11     *(char *)p = 'Z'; // c의 메모리를 강제로 덮어쓰
12
13     printf("after: s.c = %c\n", s.c);
14     return 0;
15 }
16
```

문제 12 출력 디버그 콘솔 터미널 포스트 1

```
• jungwonjun@DESKTOP-PUJ8603: /mnt/c/Users/JungwonJun/26633/ropas/SnT/260710$ gcc struct_demo.c -o struct_demo && ./struct_demo
before: s.c = A
after: s.c = Z
```

합성 시 메모리 관리

- 실제 C 언어에서는 주소 연산을 통해 인접한 메모리를 마구 참조 가능하다.
- 특히 struct를 보면 적당히 포인터 + 1 연산을 이용해 인접 변수에 접근 가능.
- CIL, 혹은 CIL--에서는 모든 변수가 외딴섬으로 존재해, 직접 그 변수를 통하지 않으면 해당 메모리 위치를 참조할 수 없게 한다.
(포인터 제외)

합성 시 메모리 관리

- 실제 C 언어에서는 주소 연산을 통해 인접한 메모리를 마구 참조 가능하다.
- 특히 struct를 보면 적당히 포인터 + 1 연산을 이용해 인접 변수에 접근 가능.
- CIL, 혹은 CIL--에서는 모든 변수가 외딴섬으로 존재해, 직접 그 변수를 통하지 않으면 해당 메모리 위치를 참조할 수 없게 한다. (포인터 제외)
- 따라서 어느 정도는 $\{x \mapsto 1, y \mapsto 2\}$ 이런 구조로 메모리를 생각할 수 있다.

stack frame 관리

- 메모리에는 크게 heap, stack, global/static이 있다.

stack frame 관리

- 메모리에는 크게 heap, stack, global/static이 있다.
- 이 중 일단 stack만 보기로 하자. 다음 간단한 함수를 보자.

```
int f(int x) {  
    if (x == 0) return 0;  
    return f(x - 1);  
}
```

stack frame 관리

- 메모리에는 크게 heap, stack, global/static이 있다.
- 이 중 일단 stack만 보기로 하자. 다음 간단한 함수를 보자.

```
int f(int x) {  
    if (x == 0) return 0;  
    return f(x - 1);  
}
```

- main에서 f(2)를 호출했을 때의 증명나무를 생각해 보자.

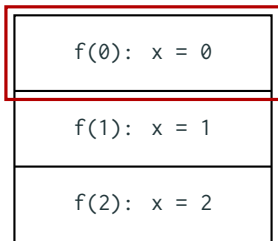
그렇지만 모든 frame을 보기엔 너무 크다

- 마지막 $f(0)$ 이 호출되었을 때, 메모리 상태는 다음과 같다.

$f(0): x = 0$
$f(1): x = 1$
$f(2): x = 2$

그렇지만 모든 frame을 보기엔 너무 크다

- 마지막 $f(0)$ 이 호출되었을 때, 메모리 상태는 다음과 같다.



- 그렇지만, 중요한(당장 봐야 하는)건 top frame뿐이다.

그렇지만 모든 frame을 보기엔 너무 크다

```
{x |-> 0 | x |-> 1 | x |-> 2} |- x == 0    {x |-> 0 | x |-> 1 | x |-> 2} |- 0 == 0 => true
-----
{x |-> 0 | x |-> 1 | x |-> 2} |- x == 0 => true
-----
{x |-> 0 | x |-> 1 | x |-> 2} |- if (x == 0) return 0; return f(x - 1); => return 0
-----
{x |-> 1 | x |-> 2} |- f(0) => 0
-----
...
```

스택의 모든 프레임을 들고다니는 경우

그렇지만 모든 frame을 보기엔 너무 크다

```

{x |-> 0} |- x == 0    {x |-> 0} |- 0 == 0 => true
-----
{x |-> 0} |- x == 0 => true
-----
{x |-> 0} |- if (x == 0) return 0; return f(x - 1); => return 0
-----
{x |-> 1} |- f(0) => 0
-----
...
  
```

top 프레임만 들고다니는 경우

그렇지만 모든 frame을 보기엔 너무 크다

```
{x |-> 0} |- x => 0    {x |-> 0} |- 0 == 0 => true
-----
{x |-> 0} |- x == 0 => true
-----
{x |-> 0} |- if (x == 0) return 0; return f(x - 1); => return 0
-----
{x |-> 1} |- f(0) => 0
-----
...
```

top 프레임만 들고다니는 경우

- 포인터를 인자로 받으면 이전 frame에 간섭이 가능하지만, 그것도 결국 따로 접근 가능한 location으로 두고, 조립할때 location을 연결하면 된다.

결국 메모리 관리는

- 이제 합성에 필요한 메모리는 다시 $\{x \mapsto 1, y \mapsto 2\}$ 꼴이므로,

결국 메모리 관리는

- 이제 합성에 필요한 메모리는 다시 $\{x \mapsto 1, y \mapsto 2\}$ 꼴이므로,
- (가능한 변수 이름(id) 목록, 가능한 값 목록)만 있으면 유한하게 메모리를 만들 수 있다.

결국 메모리 관리는

- 이제 합성에 필요한 메모리는 다시 $\{x \mapsto 1, y \mapsto 2\}$ 꼴이므로,
- (가능한 변수 이름(id) 목록, 가능한 값 목록)만 있으면 유한하게 메모리를 만들 수 있다.
- completeness 공격의 종료를 보장하기 위해서는 증명나무의 크기 증가와 함께 변수 이름, 가능한 값의 범위를 점차 넓혀가면 된다.

앞으로 할 일

앞으로 할 일

■ 지금까지 한 일

- ▶ syntax, semantics(big-step), interpreter등 기본적인 것들 구현

앞으로 할 일

■ 지금까지 한 일

- ▶ syntax, semantics(big-step), interpreter등 기본적인 것들 구현

■ 앞으로 할 일

- ▶ bottom-up 기반으로 합성 시작
- ▶ 각종 가지치기(pruning), 휴리스틱을 통한 시간 단축
- ▶ 실행 중 AI 가이드?

앞으로 할 일

■ 지금까지 한 일

- ▶ syntax, semantics(big-step), interpreter등 기본적인 것들 구현

■ 앞으로 할 일

- ▶ bottom-up 기반으로 합성 시작
- ▶ 각종 가지치기(pruning), 휴리스틱을 통한 시간 단축
- ▶ 실행 중 AI 가이드?

■ 걱정거리

- ▶ fibonacci.c만 해도 크기가 매우매우 큼.

앞으로 할 일

■ 지금까지 한 일

- ▶ syntax, semantics(big-step), interpreter등 기본적인 것들 구현

■ 앞으로 할 일

- ▶ bottom-up 기반으로 합성 시작
- ▶ 각종 가지치기(pruning), 휴리스틱을 통한 시간 단축
- ▶ 실행 중 AI 가이드?

■ 걱정거리

- ▶ fibonacci.c만 해도 크기가 매우매우 큼.
- ▶ 재귀적으로 합성할 것/아닌 것을 구분해서 쉬운 step은 바로바로 진행되게끔 하는 전략이 필요할 듯

앞으로 할 일

■ 지금까지 한 일

- ▶ syntax, semantics(big-step), interpreter등 기본적인 것들 구현

■ 앞으로 할 일

- ▶ bottom-up 기반으로 합성 시작
- ▶ 각종 가지치기(pruning), 휴리스틱을 통한 시간 단축
- ▶ 실행 중 AI 가이드?

■ 걱정거리

- ▶ fibonacci.c만 해도 크기가 매우매우 큼.
- ▶ 재귀적으로 합성할 것/아닌 것을 구분해서 쉬운 step은 바로바로 진행되게끔 하는 전략이 필요할 듯
- ▶ 재귀호출에서의 program size 역전 문제 해결

size 역전 현상

- 원래 증명나무 합성은 (prog, proof) 크기 pair를 기준으로 작은 것부터 조립식으로 만들었음

```

-----
(1, 1) {x: 1} |- x => 1      (1, 1) {x: 1} |- 1 => 1
-----
(3, 3) {x: 1} |- (x < 1) => 0
-----
(2, 2) {} |- 1 => 1
-----
(2, 2) {} |- x := 1 => {x: 1}
-----
(11, 7) {} |- x := 1;      => {x: 1}
                    while (x < 1)
                    x := (x + 1)
    
```


size 역전 현상

- 원래 증명나무 합성은 (prog, proof) 크기 pair를 기준으로 작은 것부터 조립식으로 만들었음

```

-----
(1, 1) {} |- 1 => 1
-----
(2, 2) {} |- x := 1 => {x: 1}
-----
(11, 7) {} |- x := 1;
               while (x < 1)
                 x := (x + 1)
               => {x: 1}

-----
(1, 1) {x: 1} |- x => 1      (1, 1) {x: 1} |- 1 => 1
-----
(3, 3) {x: 1} |- (x < 1) => 0
-----
(8, 4) {x: 1} |- while (x < 1) => {x: 1}
                  x := (x + 1)
-----

```

- 크기 순서를 대각선($1, 1 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 3, 1 \rightarrow \dots$)으로 잡으면 모든 증명나무를 빠짐없이 탐색 가능

size 역전 현상

- 원래 증명나무 합성은 (prog, proof) 크기 pair를 기준으로 작은 것부터 조립식으로 만들었음

```
-----  
(1, 1) {x: 1} |- x => 1      (1, 1) {x: 1} |- 1 => 1  
-----  
-----  
(3, 3) {x: 1} |- (x < 1) => 0  
-----  
-----  
(8, 4) {x: 1} |- while (x < 1) => {x: 1}  
                  x := (x + 1)  
-----  
-----  
(11, 7) {} |- x := 1;      => {x: 1}  
                while (x < 1)  
                x := (x + 1)
```

- 크기 순서를 대각선($1, 1 \rightarrow 2, 1 \rightarrow 2, 2 \rightarrow 3, 1 \rightarrow \dots$)으로 잡으면 모든 증명나무를 빠짐없이 탐색 가능
- 그런데, 함수 call을 하는 순간 부분나무가 전체 나무보다 크기가 커지는 문제가 발생!

감사합니다